TRIBHUWAN UNIVERSITY

INSTITUTE OF ENGINEERING

KATHMANDU ENGINEERING COLLEGE

KALIMATI, KATHMANDU

Minor Project Mid Defense Report

On

Tribal: Find Your Tribe

[Code Number: ENCT354]

BY

Sampada Dahal	[KAT080BCT067]
Selina Pageni	[KAT080BCT073]
Shlesha Aryal	[KAT080BCT075]
Sukriti Dhakal	[KAT080BCT083]

TO

DEPARTMENT OF COMPUTER ENGINEERING

KATHMANDU, NEPAL

June 2026

Acknowledgement

We would like to express our sincere gratitude to **Mr. Sudeep Shakya**, Head of the Department of Computer Engineering at Kathmandu Engineering College, for his leadership and support. We are also highly grateful to our Year Coordinator, **Mr. Nabin Neupane**, and our Project Coordinator, **Ms. Krista Byanju**, for their invaluable guidance, administrative support, and continuous encouragement during the formulation of this minor project proposal.

Furthermore, we extend our thanks to our project supervisor and the faculty members of the Department of Computer Engineering for their constructive feedback and expertise. Finally, we are thankful to our peers and colleagues who provided valuable suggestions that helped refine our project design and methodologies.

Abstract

Finding compatible roommates and meeting like-minded people in a new city is a problem that many people face today. Most of the existing platforms either focus only on roommate searching or only on social activities. There is no single platform that can help users find roommates, join activities, and build a trusted community all at once. TRIBAL is a mobile application built using Flutter that tries to solve this problem. The application allows users to find compatible roommates based on their lifestyle preferences, find activity partners, create and join local events, and chat with other users. The platform also includes a compatibility scoring algorithm that matches users based on factors like budget, sleep schedule, cleanliness habits, pet preferences, and other lifestyle-related questions collected during onboarding. The backend of the application is built using Django and uses PostgreSQL as the database. Authentication is handled using JWT tokens and real-time chat is being implemented using Django Channels with WebSockets.

Keywords: Roommate Matching, Compatibility Scoring, Activity Partner Discovery, Mobile Application, Flutter, Django, PostgreSQL, Real-Time Chat, Community Building, Lifestyle Preferences

Table of Contents

Acknowledgement ii

Abstract iii

Table of Contents iv

List of Figures vi

List of Tables vii

Abbreviations viii

1 Introduction 1

1.1 Background Theory 1

1.2 Problem Statements 1

1.3 Objectives 2

1.4 Scope and Applications 2

2 Literature Review 4

2.1 Roomster 4

2.2 Bumble BFF 4

2.3 Meetup 4

2.4 Facebook Groups 5

2.5 Roomi 5

2.6 SpareRoom 5

2.7 Summary of Limitations 5

2.8 How Tribal is different 6

3 Methodology 7

3.1 Process Model 7

3.2 System Block Diagram 8

3.3 Algorithm 9

3.3.1 Roommate Compatibility Scoring Algorithm 9

3.3.2 Deal Breakers 10

3.3.3 Final Score Calculation 11
3.4 Flowchart 12
3.5 UML Diagrams 12
3.6 Tools Used 13

4 Epilogue **14**
4.1 Tasks Completed 14
4.2 Remaining Tasks 14
4.3 Updated Gantt Chart 15

References **16**

Bibliography **17**

Screenshots **18**

List of Figures

3.1 Agile Process Model 8
3.2 System Block Diagram 8
3.3 System Flowchart 12
3.4 UML Diagram 12

1 TRIBAL Application Screenshots (Part 1) 18
2 TRIBAL Application Screenshots (Part 2) 19
3 TRIBAL Application Screenshots (Part 3) 19

List of Tables

3.1 Compatibility Scoring Weights 10
3.2 Deal Breaker Rules 11
3.3 Tools and Technologies Used 13

4.1 Gantt Chart – Updated Schedule 15

Abbreviations

API	Application Programming Interface
Dart	Dart Programming Language
Django	Django Web Framework
Figma	Figma Design Tool
Flutter	Flutter UI Framework
Git	Git Version Control System
GitHub	GitHub Code Hosting Platform
JWT	JSON Web Token
MVC	Model-View-Controller
PostgreSQL	PostgreSQL Database
Postman	Postman API Testing Tool
Python	Python Programming Language
REST	Representational State Transfer
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VS Code	Visual Studio Code

Chapter 1 Introduction

1.1 Background Theory

In recent years, the increasing number of students and working professionals moving to new cities has created a growing need for platforms that help users find compatible roommates and build social connections. Existing social media and roommate-finding applications either lack compatibility-based matching or focus only on basic profiles, leaving a gap in the market.

TRIBAL is a cross-platform mobile application developed using Flutter and Django to address this problem. It enables users to create profiles, answer lifestyle-based onboarding questions, and find compatible roommates and activity partners through a weighted compatibility scoring algorithm. The application also supports event creation, participation, and real-time chat between matched users.

The weighted compatibility scoring approach was chosen over machine learning because it is transparent, easier to implement within the scope of a minor project, and provides meaningful matches based on users' preferences and interests.

1.2 Problem Statements

People who want to find others with shared interests often have no proper platform to do so. General social media platforms are too broad and not designed for this kind of need. Activity-specific platforms like Meetup are not widely used in Nepal and do not support roommate finding at all.

The main problems that TRIBAL tries to address are:

- Finding compatible roommates and building a social circle is challenging for students and young professionals who move to new cities, as they often rely on informal methods such as friends or social media groups.
- Existing platforms do not provide a single solution for roommate matching, activity partner discovery, and community building, while also overlooking important lifestyle compatibility factors.

- TRIBAL addresses these challenges by offering a unified platform with compatibility-based roommate matching, activity partner discovery, event participation, and secure in-app chat.

TRIBAL aims to solve these problems by providing a single mobile platform where users can find compatible roommates, discover activity partners, join events, and chat with matches in a safe environment.

1.3 Objectives

The main objectives of this project are:

- To design and develop a Flutter-based mobile application with a Django and PostgreSQL backend that enables users to find compatible roommates and activity partners through secure authentication, profile management, and a weighted compatibility matching algorithm.
- To implement core social features, including real-time chat, event creation and participation, and interest-based activity discovery, while ensuring secure communication and data management.

1.4 Scope and Applications

The scope of this project as a minor project includes the following features:

- Develop a cross-platform Flutter application that enables users to find compatible roommates and activity partners through secure authentication, profile management, and preference-based matching.
- Provide essential social features, including real-time chat, event creation and participation, and interest-based activity discovery to help users build meaningful connections.
- Address the challenges of finding roommates and social networks, particularly for students and newcomers in Nepal, while serving as a scalable prototype for future enhancements such as verified profiles and location-based recommendations.

For the purpose of this minor project, the focus is on completing the core features

including the compatibility matching system, real-time chat, and event module. The application is not intended for commercial deployment at this stage, but it demonstrates a working prototype of the complete system.

Chapter 2 Literature Review

Before starting the design of TRIBAL, the team looked at some existing applications that solve similar problems. This helped the team understand what features are already available, what is missing, and how TRIBAL can be different from these platforms. This chapter looks at a few popular apps that are related to roommate finding and activity matching, and explains their limitations.

2.1 Roomster

Roomster is a roommate and room-finding platform that is mainly used in the United States and a few other countries. It allows users to post listings for rooms and search for roommates based on location and budget. However, Roomster does not have any real compatibility matching system. It mostly works like a classified listing site where users browse profiles and contact each other manually. There is no scoring system to tell a user how well they might get along with a potential roommate based on lifestyle habits like cleanliness, sleep schedule, or smoking

2.2 Bumble BFF

Bumble BFF is a friend-finding mode of the popular Bumble dating app. It uses a swipebased system where users can like or pass on other profiles to find friends with similar interests. While this works well for general friend matching, it is not designed for roommate finding. Bumble BFF also does not collect detailed lifestyle information like budget range or sleep schedule, which are important factors when two people are going to live together.

2.3 Meetup

Meetup is a platform mainly used to organize and join group activities and events based on shared interests. It works well for finding activity partners but does not have any roommate matching feature at all. Meetup is also not very popular in Nepal, and most local activity groups still rely on Facebook groups or word of mouth instead.

2.4 Facebook Groups

Facebook Groups are widely used in Nepal for both roommate searching and activity planning. Many people post in groups like "Room for Rent Kathmandu" or "Hiking Buddies Nepal" to find roommates or activity partners. The biggest problem with this method is that there is no structured system behind it. Anyone can post anything, there is no compatibility check, and there is no way to verify whether the listed information is true. This often leads to mismatched roommates and unsafe meetups with strangers.

2.5 Roomi

Roomi is another roommate-finding application that allows users to filter listings by location, price, and a few basic preferences. It is more focused on the United States market and is not commonly used in Nepal. Like Roomster, Roomi does not use any weighted scoring system, so users still have to manually go through profiles and decide compatibility on their own.

2.6 SpareRoom

SpareRoom is a roommate listing platform used mostly in the United Kingdom. It allows detailed filtering of listings, such as gender preference and smoking status, but the matching is still based on simple filters rather than an actual compatibility score. There is no algorithm that combines multiple lifestyle factors into a single match percentage.

2.7 Summary of Limitations

After looking at these platforms, the team noticed a few common limitations:

- Most platforms either focus only on roommate listing or only on activity and event matching, but not both.
- None of these apps use a proper compatibility scoring system that takes multiple lifestyle factors into account at once.
- Apps that are popular internationally, like Roomster, Roomi, and SpareRoom, are not commonly used or trusted in Nepal.

- Local methods like Facebook groups do not provide any safety or compatibility checks, which can lead to bad roommate experiences.
- There is no platform that allows users to both find a compatible roommate and discover local activity partners from one single application.

2.8 How Tribal is different

TRIBAL tries to solve these problems by combining roommate matching and activity and event discovery into a single mobile application. Instead of relying on manual browsing, TRIBAL uses a weighted compatibility scoring algorithm that compares onboarding answers from two users and generates a compatibility score out of 100. This score considers multiple lifestyle factors together, such as cleanliness, budget, sleep schedule, noise level, smoking, guests, interests, study habit, food, and drinking preference, instead of just one or two filters. By keeping the matching system rule-based instead of using machine learning, TRIBAL is able to give users a transparent and explainable match score. A user can see why their compatibility score with another person is high or low, which is something most existing platforms do not provide. This approach is also more practical to build and test within the time frame of a minor project, while still giving meaningful results to users.

Chapter 3 Methodology

3.1 Process Model

The team is following the Agile process model for the development of TRIBAL. Agile was chosen because it allows the project to be broken down into small, manageable parts called sprints, and each sprint produces a working piece of the application. This is useful for a minor project because requirements can be refined as the team learns more, and progress can be shown at regular intervals like this mid-term defense. The project has been divided into the following sprints:

- Sprint 1: Project setup, requirement analysis, and UI/UX wireframing in Figma.
- Sprint 2: Flutter onboarding flow, authentication screens (Login and Signup), and post-signup profile completion screens.
- Sprint 3: Django backend setup, PostgreSQL database design, and JWT-based authentication APIs.
- Sprint 4: Design and implementation of the roommate compatibility scoring algorithm (current sprint).
- Sprint 5: Real-time chat system using Django Channels and WebSockets.
- Sprint 6: Event creation and joining module.
- Sprint 7: Integration testing between the Flutter frontend and Django backend.
- Sprint 8: Final testing, bug fixing, and documentation.

At the time of this report, the team has completed Sprints 1 to 4 and has started work on Sprint 5.

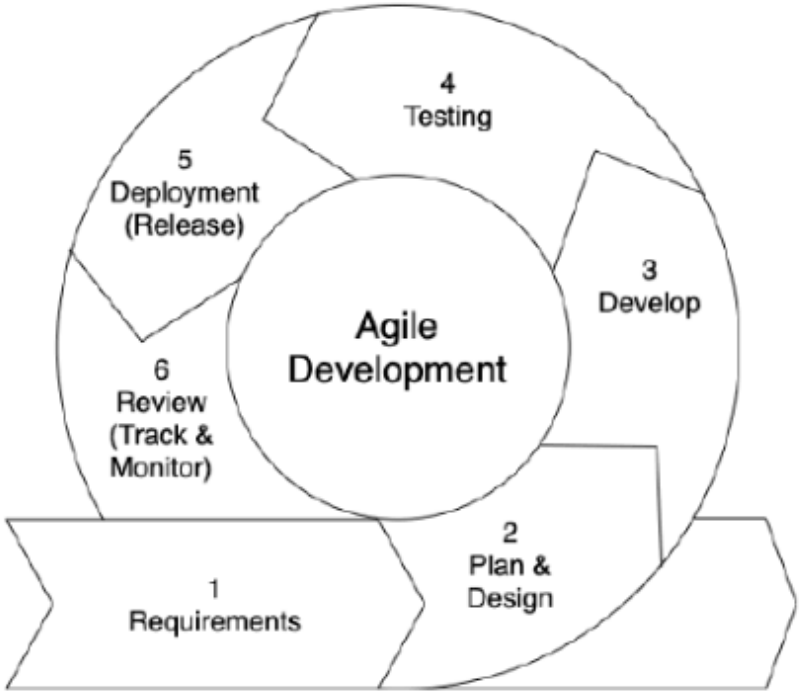


Figure 3.1: Agile Process Model

3.2 System Block Diagram

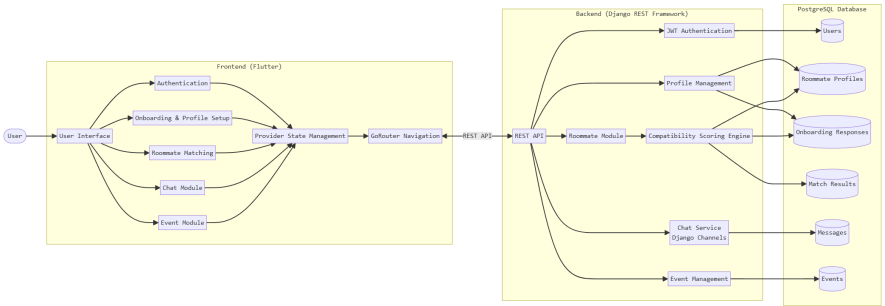


Figure 3.2: System Block Diagram

The system is divided into three main layers:

- Frontend (Flutter): Handles the user interface, including onboarding, authentication screens, and profile setup. The frontend uses Provider for state management, GoRouter for navigation, and follows an MVC-based architecture.
- Backend (Django): Exposes REST APIs for authentication, user profiles, and roommate matching. The roommate matching logic lives entirely inside a single roommate module, which keeps the matching-related code organized and separate from the rest of the backend.

- Database (PostgreSQL): Stores user accounts, roommate profile data, and onboarding responses that are later used by the scoring algorithm.

The Flutter app communicates with the Django backend through REST APIs. JWT tokens are used to keep the user logged in and to secure each API request. When a user requests roommate matches, the request goes to the roommate module, which fetches relevant profiles from the database, runs the scoring algorithm, and returns a ranked list of compatible roommates back to the app.

3.3 Algorithm

3.3.1 Roommate Compatibility Scoring Algorithm

The core feature of TRIBAL at this stage is the roommate compatibility scoring algorithm. The team decided not to use machine learning for this feature. Instead, a rule-based weighted scoring system was designed. This decision was made because a rule-based system is easier to explain, easier to test, and does not need a large dataset to train on, which makes it more suitable for a minor project than a machine learning model. The algorithm compares two users' onboarding answers and produces a single compatibility score between 0 and 100. The onboarding answers used in scoring include cleanliness, budget, sleep schedule, noise level, smoking, guests, interests, study habit, food preference, and drinking habit. The pet preference field is collected during onboarding and stored in the profile, but it is not currently used in the scoring calculation. Each factor is given a weight depending on how much it usually affects roommate compatibility in real life. For ex le, cleanliness and budget are given higher weights because mismatches in these areas are more likely to cause conflict, while smaller lifestyle preferences are given lower weights.

Table 3.1 below shows the weight distribution used in the scoring algorithm.

Factor	Weight (%)
Cleanliness	15
Budget	15
Sleep Schedule	12
Noise Level	10
Smoking	10
Guests	8
Interests	10
Study Habit	8
Food Preference	7
Drinking	5

Table 3.1: Compatibility Scoring Weights

The general formula used to calculate the final compatibility score is:

$$\text{Final Score} = \sum (\text{Factor Match Value} \times \text{Factor Weight}) \tag{3.1}$$

Where Factor Match Value is a number between 0 and 1 that represents how closely two users' answers match for a particular factor, and Factor Weight is the percentage weight assigned to that factor as shown in Table 3.1. For factors with a small number of fixed options, such as smoking or sleep schedule, the match value is calculated using a simple rule: if both users selected the same option, the match value is 1; if their answers are close but not the same (for ex le, "early riser" and "no preference"), a partial match value such as 0.5 is given; and if the answers directly conflict (for ex le, "smoker" matched with "non-smoker, no smoking allowed"), the match value is set to 0. For factors that involve multiple selections, such as interests, the match value is calculated as the number of common interests divided by the total number of unique interests selected by both users combined. This gives a percentage overlap between the two users' interests. For numeric factors like budget, the match value decreases as the difference between the two users' stated budgets increases. If the difference is within an acceptable range, the match value stays close to 1. If the difference is very large, the match value moves closer to 0.

3.3.2 Deal Breakers

Apart from the weighted scoring, the algorithm also includes a deal breaker check. Some lifestyle differences are considered serious enough that they should not simply lower the

score a little, but should instead strongly affect or override the final match. For ex le, if one user is a heavy smoker and the other has explicitly said they cannot live with a smoker at all, this is treated as a deal breaker rather than just a normal mismatch. Table 3.2 lists the deal breaker rules currently used in the algorithm.

Condition	**Effect on Score**
Smoking conflict (smoker vs. strictly non-smoking)	Score capped at a low maximum
Extreme budget mismatch (outside acceptable range)	Score capped at a low maximum
Opposite cleanliness extremes (very messy vs. very strict)	Score significantly reduced

Table 3.2: Deal Breaker Rules

When a deal breaker condition is detected, the algorithm does not let the final score go above a fixed low ceiling, even if all other factors match well. This makes sure that users are not shown as highly compatible with someone they have a fundamental lifestyle conflict with.

3.3.3 Final Score Calculation

Once all the individual factor match values are calculated and weighted, they are added together to get the final compatibility score. If a deal breaker is triggered, the score is capped according to the deal breaker rule before being returned to the user. The final score is then rounded to the nearest whole number and shown to the user as a percentage match out of 100. This score is what the user sees in the app when browsing potential roommates, helping them quickly understand how compatible they may be with someone before starting a conversation.

3.4 Flowchart

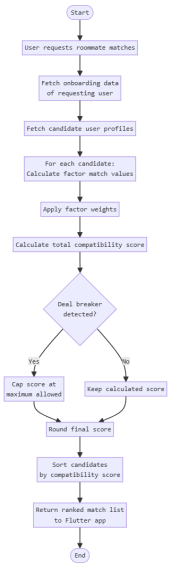


Figure 3.3: System Flowchart

The flowchart illustrates the step-by-step operational flow of the TRIBAL system.

3.5 UML Diagrams

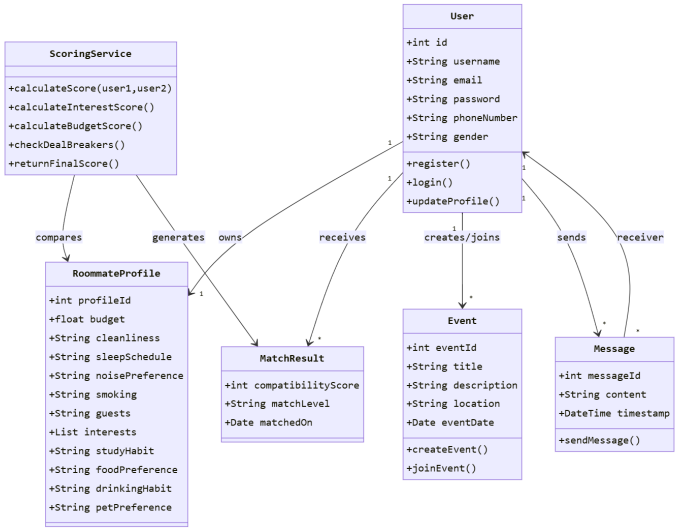


Figure 3.4: UML Diagram

The UML diagram provides an object-oriented view of the system's architecture, showing key classes and their relationships.

3.6 Tools Used

The following tools and technologies are being used in the development of TRIBAL:

Category	**Tool / Technology**
Frontend Framework	Flutter
Frontend Language	Dart
State Management	Provider
Navigation	GoRouter
Backend Framework	Django
Backend Language	Python
Database	PostgreSQL
Authentication	JWT
Real-Time Communication	Django Channels, WebSockets
Version Control	Git, GitHub
Code Editor	Visual Studio Code
Mobile Testing	Android Studio
API Testing	Postman
UI/UX Design	Figma

Table 3.3: Tools and Technologies Used

Flutter was chosen for the frontend because it allows the team to write one codebase that works on both Android and iOS. Django was chosen for the backend because it comes with many built-in features like an ORM and an admin panel, which makes development faster for a project with this scope. PostgreSQL was chosen as the database because it works well with Django and can reliably handle relational data like user profiles and onboarding responses.

Chapter 4 Epilogue

4.1 Tasks Completed

So far, the team has completed the following tasks:

- UI design for onboarding, login, signup, and profile setup screens in Flutter.
- Implementation of the three-slide onboarding screen with skip and next functionality.
- Animated loading screen shown during navigation between major flows.
- Django project setup with PostgreSQL as the database.
- JWT-based authentication APIs for login and signup.
- Database models for User and RoommateProfile.
- Design of the roommate compatibility scoring algorithm with weighted factors and deal breaker rules.
- Manual API testing using Postman.
- GitHub repository setup for version control and team collaboration.

4.2 Remaining Tasks

The following tasks are still remaining and will be completed in the upcoming sprints before the final defense:

- Implementation of the scoring logic inside the roommate module (models, scoring, services, views, serializers, urls, admin).
- Initial unit tests for the scoring function.
- Integration of the roommate matching API with the Flutter frontend, so that real compatibility scores are shown in the app instead of placeholder data.
- Implementation of the real-time chat system using Django Channels and WebSockets.
- Chat screens in Flutter for one-to-one conversations between matched users.
- Event creation and joining module, including event listing and filtering screens.
- Notification system for new matches and messages.
- Full integration testing between the Flutter frontend and the Django backend.

- Final round of bug fixing and performance testing.
- Preparation of the user manual and final project documentation.
- Deployment of the backend and preparation for the final presentation.

4.3 Updated Gantt Chart

The original time schedule has been adjusted slightly based on the actual pace of development so far. The updated schedule below shows the completed sprints and the plan for the remaining weeks leading up to the final defense.

Task	**Status**
Sprint 1: Requirement Analysis & Wireframing	Completed
Sprint 2: Flutter Onboarding & Auth Screens	Completed
Sprint 3: Django Backend & Database Setup	Completed
Sprint 4: Roommate Compatibility Algorithm	Completed
Sprint 5: Real-Time Chat System	In Progress
Sprint 6: Event Module	Planned
Sprint 7: Frontend-Backend Integration Testing	Planned
Sprint 8: Final Testing & Documentation	Planned

Table 4.1: Gantt Chart – Updated Schedule

The team is currently on Sprint 5. Based on the progress made so far, the team is confident that the remaining sprints can be completed within the original 16-week timeline, with some buffer time set aside before the final defense for bug fixing and documentation.

References

[1] Google, *Flutter documentation*, 2024.

[2] D. S. Foundation, *Django documentation*, 2024.

[3] Encode, *Django rest framework documentation*, 2024.

[4] D. S. Foundation, *Django channels documentation*, 2024.

[5] P. G. D. Group, *Postgresql documentation*, 2024.

[6] Auth0, *Json web tokens introduction*, 2024.

[7] pub.dev, *Provider package documentation*, 2024.

[8] pub.dev, *Gorouter package documentation*, 2024.

Bibliography

[1] Roomster. “Roomster. ”https://www.roomster.com/.

[2] Bumble. “Bumble bff. ”https://bumble.com/bff.

[3] Meetup. “Meetup. ”https://www.meetup.com/.

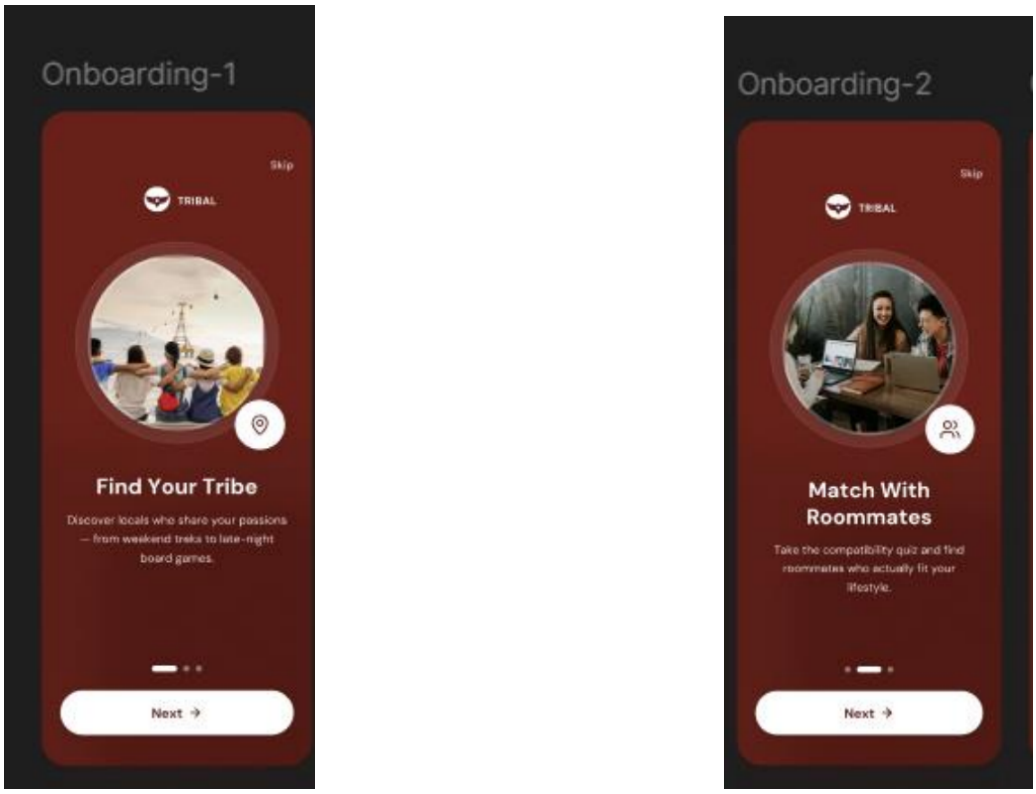
[4] Roomi. “Roomi. ”https://www.roomiapp.com/.

[5] SpareRoom. “Spareroom. ”https://www.spareroom.com/.

[6] Postman. “Postman api platform. ”https://www.postman.com/.

[7] Figma. “Figma design tool. ”https://www.figma.com/.

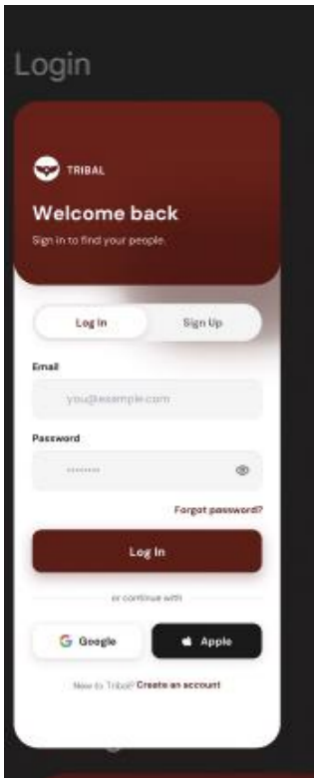
Screenshots



(a) Screenshot 1 (b) Screenshot 2

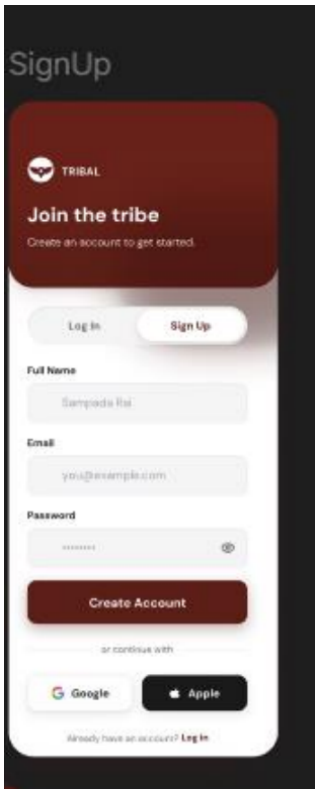
Figure 1: TRIBAL Application Screenshots (Part 1)

(a) Screenshot 3

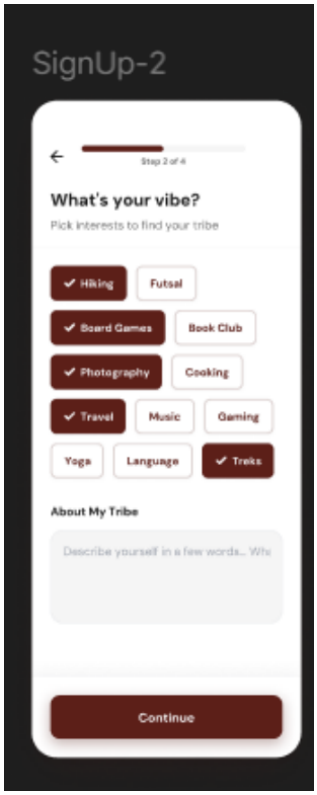


(b) Screenshot 4

Figure 2: TRIBAL Application Screenshots (Part 2)



(a) Screenshot 5



(b) Screenshot 6

Figure 3: TRIBAL Application Screenshots (Part 3)